

Scenario Investigations — k -arm coin-toss, pick-and-place, quadruped

Aiko (Claude Code) for Dr. Cs. Hajdu

2026-06-28

Executive summary

Three reinforcement-learning scenarios in the HyMeKo codebase were audited for the gap between what is implemented and what the designed-control thesis still needs. The two-arm Galambos coin-toss and the seven-phase pick-and-place pipeline both work as scripted demonstrators, but learned policies stall on each — and in both cases the stall is a *reward decoupling* (a shortcut term the policy exploits) rather than a controller limitation. The quadruped is a genuine MuJoCo robot that already jumps and partially walks; its missing piece for the thesis is a standing/balance task, and its nearest watchable artifact is the planar-walker checkpoint.

1 k -arm finger coin-toss

The two-arm “Galambos” coin-toss is built and functional: a top-down, two-finger planar grasp with a CTDE (centralised-training, decentralised-execution) wrapper and a passing test suite. The collaborative extension to $k > 2$ arms is, by contrast, almost entirely on paper. The wrapper that exists is hardcoded for $k = 2$: it partitions agents by actuator name suffixes (`_left` / `_right`), so it does not generalise as written.

Component	State
PlanarGraspEnv (<code>env/planar_grasp.env.py</code>), 2-arm top-down two-finger grasp	BUILT
collaborative.py: CollaborativeGalambos, CTDEActorCritic	BUILT (hardcoded)
partitions on actuator names <code>_left</code> / <code>_right</code>	
<code>exp_collaborative.py</code> + <code>test_collaborative.py</code> (5 tests)	PASSING
Dual-discriminator + k -arm scaling (plan 2026-06-24-kato-collab-dual-discriminator)	PLAN ONLY
Arm template <code>planar_finger.hymeko</code>	NOT STARTED
k -agent CTDE off-policy trainer (<code>marl.py</code>)	NOT STARTED
Hybrid HSiKAN + MLP + CliffordFIR backbone	NOT STARTED
Weighted-incidence reader	NOT STARTED

Table 1: k -arm coin-toss component status.

Key finding. The load-bearing blocker is that `PlanarGraspEnv` does *not* parametrize the arm count. Everything above it — dual-discriminator, MARL trainer, weighted-incidence reader — sits on top of a working k -arm environment that does not yet exist.

Next action. Create `data/robotics/planar_finger.hymeko` (a single two-link finger template) → instantiate k copies in `galambos_planar.hymeko` → add an `n_arms` parameter to `PlanarGraspEnv` and generalize the left/right partition → add a $k = 3$ rollout test. Estimated ≈ 3 hours.

2 Pick-and-place

A seven-phase, state-gated scripted pipeline works end-to-end; learned policies stall at the same wall. The cause is a reward decoupling, not a controller limit. The phases (`env/pick_place_env.py`) are:

reach → descend → grasp → settle (12-step dwell) → lift → transport → place.

Term	Weight	Role
approach	1.0	dense descend, saturates
contact	0.5	binary finger touch
lift	5.0	CAPPED at <code>lift_thresh</code> 0.035 m
place	1.0	xy distance, GATED on lifted > 0.02
placed	20.0	sparse success
noground	−2.0	approach-collision safety
nonudge	−3.0	pre-grasp; latches off after grasp

Table 2: Pick-and-place reward terms.

Measured. Scripted expert: 10/10 grasp, 9/10 lift, 9/10 place (FANUC). BC: 0% place (covariate shift over a 620-step horizon). PPO warm-start: 7/8 contact, ≈ 1.2 cm lift, 0% place. Curriculum PPO: 0% place at all difficulties.

Key finding (the bottleneck). The `lift` term is **CAPPED** at 0.035 m, so a learned policy hits the cap and *hovers* — exploiting +0.175/step — and never explores the full lift→place. Meanwhile `place` is under-shaped: only xy distance, gated, with the final descent unrewarded except for the sparse +20. This is the same “exploit a shortcut” pattern as the coin-toss knock-not-grasp, and the recent finding that *low reward-conflict drives better behaviour* applies directly: the seven terms have decoupling gaps in which `lift` masks `place`.

Next action. UNCAP the `lift` term (reward full height) + raise `place` weight 1.0 → 3–5 + add an intermediate place-proximity shaping term gated during descent; then re-test BC→PPO→TD3. Reward edits go in `data/robotics/pick_place_task.hymeko` per the project rule.

3 Quadruped

A real MuJoCo quadruped exists and jumps. Planar walking is a partial win; standing/balance — the task the designed-control thesis needs — is not yet built.

Key finding. The quadruped is genuinely a real MuJoCo robot — not an abstract linear plant. The “quadruped” in `structural_control_loop.py` is only a *planned* future ladder rung. The nearest watchable artifact today is the planar-walker checkpoint.

Component	State	Evidence
<code>quadruped.hymeko</code> descriptor	BUILT	torso + 4 two-link legs, 8 DOF, 9 vertices
<code>QuadrupedGoalEnv</code> (<code>env/quadruped_env.py</code>)	BUILT	3 base modes (free/planar/fixed); 12 tests pass; 200 steps < 2 s
Jumping (free base)	DONE	+0.69 m above 0.42 m rest; PPO 80 iters
Planar walking	PARTIAL	+0.47 m forward; ckpt <code>checkpoints/walker_planar_hsikan.pt</code> ; just under $x > 0.6$ success threshold
Free-base 3D walking	FAILED	flails; 0% goal
Standing/balance task	NOT STARTED	needed for <code>exp_designed_control</code> rung 2 (registered <code>_PLANTS["quadruped"]</code> but unused)
3D-walker constraint curriculum	PLAN ONLY	plan 2026-06-22-3d-walker-constraints

Table 3: Quadruped component status.

Next action (quickest visual, ≈ 5 min). Load `checkpoints/walker_planar_hsikan.pt` into an HSiKAN policy on `QuadrupedGoalEnv(base='planar')` and render a GIF of the ≈ 0.47 m walk. For the designed-control thesis: build a standing/balance reward (stay upright, minimize base drift) so the kinematic-vs-designed-hypergraph comparison has a balancing task.